Quantifying Information Flow with Entropy

Module Q32 – AI Course 2026

Kukla

Anchor: L2.1 ▪ Week 4

The question hides three questions

"Can we quantify the information flowing through a model?"

1. Output uncertainty — entropy of $p(x_t \mid x_{<t})$. *exact, cheap*

The question hides three questions

"Can we quantify the information flowing through a model?"

1. Output uncertainty — entropy of $p(x_t \mid x_{<t})$. *exact, cheap*
2. Compression quality — bits/token = cross-entropy. *= the training loss*

The question hides three questions

"Can we quantify the information flowing through a model?"

1. Output uncertainty — entropy of $p(x_t \mid x_{<t})$. *exact, cheap*
2. Compression quality — bits/token = cross-entropy. *= the training loss*
3. Internal flow — mutual information between activations. *careful!*

The question hides three questions

"Can we quantify the information flowing through a model?"

1. Output uncertainty — entropy of $p(x_t \mid x_{<t})$. *exact, cheap*
2. Compression quality — bits/token = cross-entropy. *= the training loss*
3. Internal flow — mutual information between activations. *careful!*

Yes for 1 & 2 exactly — "yes, carefully" for 3.

Entropy: how much is the model hedging?

For next-token distribution p over vocabulary $\mathcal{V}$:

$$H(p) = -\sum_{v} p(v) \log_2 p(v) \quad \text{bits}$$

- $H = 0$: certain. $\quad H = \log_2 |\mathcal{V}| \approx 15$–$17$: uniform guess.
- A per-step read-out of confidence — *after* temperature & softmax.
- High entropy at a decision-critical token = the model is uncertain (*after calibration*, a useful abstention flag — not a correctness oracle).

Cross-entropy = the loss = compression

$$\mathrm{CE} = -\frac{1}{T}\sum_{t} \log_2 p_\theta(x_t \mid x_{<t}), \qquad \mathrm{PPL} = 2^{\mathrm{CE}}$$

$$\mathrm{CE} = \underbrace{H(p_{\text{data}})}_{\text{irreducible}} + \underbrace{D_{\mathrm{KL}}(p_{\text{data}} \| p_\theta)}_{\text{what training removes}}$$

Compression is literal

A frozen LLM used as the probability model inside an arithmetic coder beats `gzip` on text *and* images (Delétang et al. 2023). Bits/token and model quality are one axis.

Three things entropy buys you

Confidence signal

High entropy $\rightarrow$ abstain / flag hallucination (Q10–Q12).

Decoding

Temperature, top-k, nucleus, min-p are all entropy-reshaping. Watch for RL "entropy collapse."

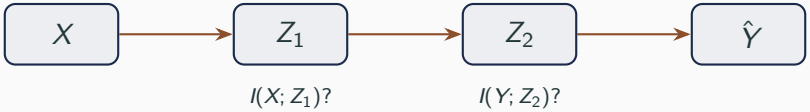
Training diagnostic

Mean entropy falls from $\sim$16 bits to a few as the model learns.

Internal flow: the seductive, dangerous part

Mutual information between a hidden rep $Z = f_\ell(X)$ and input/label:

$$I(X; Z) = H(X) - H(X \mid Z)$$



The Information Bottleneck claimed a "compression phase." It was mostly a binning artefact.

The cautionary tale

Saxe et al. 2018
The famous IB "compression phase" depends on the *nonlinearity* and the *binning*: `tanh` shows it, `ReLU` often doesn't. For a deterministic net, $I(Z;X)$ is ill-defined/infinite with continuous X (bounded by $H(X)$ when discrete) — the plotted number is a property of your *estimator*, not the network.

The estimator *is* the measurement.

What is actually safe to measure

- **Attention entropy** — weights are already a distribution. Low = "pointing", high = "averaging". Exact, no assumptions.
- **Predictive / cross-entropy** — exact, cheap, load-bearing.
- **MI between activations** — needs a principled bound (KSG, MINE, InfoNCE) and is loose above a few nats.
- **Causal flow?** Don't estimate — intervene: patch/ablate the path, measure the output change (Q14/Q27).

Takeaways

1. Predictive entropy + perplexity give you most of what you want, exactly.
2. Attention entropy is a safe internal read-out.
3. Treat any "information plane" plot as a claim about the estimator.
4. For causal flow, patch — don't estimate.

Code demo: `entropy_flow.py` computes all the safe quantities on *synthetic* distributions (numpy-only, deterministic) — same math as a real model — and shows the binning artefact live.